BETHOVEN ACHA TRAINING DELIVERY

# Demonstration guide

A browser-led walkthrough of Week 3 dead-letter recovery, Week 4 idempotency and durable admission, Week 5 platform controls, and Week 6 reversible releases.

**Run from**
`C:\Users\Full Scale\projects\Projects`

**Prepared from current source**
25 August 2026

**Week 3 API client**
Swagger UI

**Other HTTP demonstrations**
Postman desktop UI

**Interaction rule:** use Swagger for the Week 3 API calls and Postman for the Week 4 request sequence and Week 6 health checks. Terminal commands appear only where a browser client cannot control containers, run provisioning, or perform a release.

```
Swagger / Postman → API → SQL intent + Outbox → Relay → RabbitMQ → Worker + Inbox
  failure → bounded retry → DLQ → Recovery Worker → delayed replay → Completed
```

# 1. Preflight and safety boundary

## Required tools

- Docker Desktop using the Linux engine and Docker Compose v2.
- A browser for Swagger, RabbitMQ Management, and the web applications.
- Postman Desktop for the repository's Week 4 collection and release health checks.
- PowerShell for scoped container lifecycle commands; Git Bash for Week 5 and Week 6 scripts.

## Prepare local Week 3 credentials

```
Set-Location 'C:\Users\Full Scale\projects\Projects'
if (-not (Test-Path notification-api/.env)) {
  Copy-Item notification-api/.env.example notification-api/.env
}

# Open notification-api/.env in your editor and fill all four values:
# WEEK3_SQL_PASSWORD, WEEK3_RABBITMQ_USERNAME,
# WEEK3_RABBITMQ_PASSWORD, and WEEK3_JWT_SIGNING_KEY.
```

Use strong local-only values. The JWT signing key must contain at least 32 characters. The ignored `.env` file must never be committed or displayed while screen sharing.

## Start the Week 3 stack

```
$week3 = 'notification-api/compose.week3.yml'
docker compose --env-file notification-api/.env -f $week3 config --quiet
docker compose --env-file notification-api/.env -f $week3 up --build --detach
docker compose --env-file notification-api/.env -f $week3 ps
```

**Expected: SQL Server, RabbitMQ, API, Outbox Relay, primary Worker, and DLQ Recovery Worker become healthy.**

> **Never use during the demo:** `docker compose down --volumes`, RabbitMQ actions that fetch or acknowledge a message, or any screen that reveals `.env`, Vault tokens, AppRole material, or rendered secrets.

# 2. Week 3: prepare a controlled DLQ entry

The normal recovery worker must be paused so the rejected command stays visible. A deliberately named one-off primary worker enables the local failure hook without editing Compose or exposing secrets.

> **Critical:** remove the one-off poison worker before recovery. If it remains active, it will poison every replay until the independent recovery limit is exhausted.

**1** Pause the canonical consumers and start the isolated worker.

```
$poisonWorker = 'nsa-week3-poison-worker'
docker compose --env-file notification-api/.env -f $week3 stop worker dlq-recovery-worker
docker compose --env-file notification-api/.env -f $week3 run --detach --no-deps `
  --name $poisonWorker `
  -e 'RabbitMq__FailureInjectionSubject=[week3-poison]' worker
```

**2** In Docker Desktop, wait until `nsa-week3-poison-worker` reports **healthy**. Leave SQL Server, RabbitMQ, the API, and the Outbox Relay running.

**3** Open these browser tabs:

| Surface | Address | Use |
|---|---|---|
| Swagger | `http://127.0.0.1:8080/swagger` | Submit the poison job and read its SQL-backed status. |
| RabbitMQ | `http://127.0.0.1:15672` | Show the durable DLQ entry and queue counts. |
| Docker Desktop | Containers view | Show that API and Worker are separate processes. |

> Sign in to RabbitMQ Management with the local Week 3 broker credentials, but do not reveal them. Under **Queues and Streams**, confirm the main, recovery, DLQ, and parking queues exist. Do not click **Get messages**.

```
normal workers paused → isolated poison worker healthy → Swagger ready → recovery held for inspection
```

W

### BEFORE RECOVERY

Remove the one-off poison worker before starting DLQ recovery:

**docker rm -f nsa-week3-poison-worker**

# 3. Week 3: submit and inspect with Swagger

1 In Swagger, choose **Notification Service API v2**. Expand **POST /api/v2/notifications/bulk**, click **Try it out**, and leave `X-Idempotency-Key` blank. Replace `<run-id>` with a short timestamp so repeated demonstrations use distinct recipients.

```
{
  "notifications": [
    {
      "recipientEmail": "week3-admin-<run-id>@example.test",
      "channel": 4,
      "subject": "[week3-poison]",
      "body": "Recovered administrator notification.",
      "orderId": null
    },
    {
      "recipientEmail": "week3-visitor-<run-id>@example.test",
      "channel": 4,
      "subject": "[week3-poison]",
      "body": "Recovered visitor notification.",
      "orderId": null
    }
  ]
}
```

Click **Execute**. Expect `202 Accepted`. Copy the response `jobId`; also point out the `Location` and `X-Correlation-ID` response headers.

2 Expand **GET /api/v2/notifications/bulk/{jobId}**, click **Try it out**, paste the GUID, and click **Execute** again until the terminal state appears.

**Expected after three worker attempts: `DeadLettered`, with total / processed / succeeded / failed = `2 / 0 / 0 / 0`.**

3 Refresh RabbitMQ **Queues and Streams**. Show `nsa.notifications.bulk.dlq` with **Ready = 1** and **Unacked = 0**.

> **Say this:** "The primary worker exhausted a bounded retry budget, persisted the DeadLettered state, and rejected without requeue. RabbitMQ retained the command in the durable DLQ instead of losing it or looping forever."

# 4. Week 3: recover and verify with Swagger

1. Remove the injected worker first, then restore the canonical consumers.

```
docker stop $poisonWorker
docker rm $poisonWorker
docker compose --env-file notification-api/.env -f $week3 up --detach --no-deps `
  worker dlq-recovery-worker
```

2. Return to the same Swagger **GET /api/v2/notifications/bulk/{jobId}** operation. Click **Execute** until it reports `Completed`. `RecoveryPending` is a valid transient state while the recovery queue applies its configured delay.

**Expected after about five seconds: total / processed / succeeded / failed = `2 / 2 / 2 / 0`.**

3. In Swagger, expand **GET /api/v2/notifications**. Query once with the admin email and once with the visitor email used on Page 4. Each result should contain exactly one recovered in-application notification with subject `[week3-poison]`.

4. Refresh RabbitMQ Management. The main, recovery, and DLQ queues should return to zero.

```
DeadLettered → validate death metadata → RecoveryPending → confirmed publish
      → ACK DLQ copy → delay queue → normal Worker → Completed + Inbox
```

## Interrupted-demo restoration

```
docker stop nsa-week3-poison-worker 2>$null
docker rm nsa-week3-poison-worker 2>$null
docker compose --env-file notification-api/.env -f $week3 up --detach --no-deps `
  worker dlq-recovery-worker
```

> These commands target only the deliberately named one-off container and preserve the Week 3 SQL and RabbitMQ volumes.

# 5. Week 4: use Postman for durable admission

Switch from the Week 3 stack to the Week 5 platform, which contains the Week 4 durability behavior and exposes the same API address.

```
# PowerShell: stop Week 3 containers but preserve their named volumes
docker compose --env-file notification-api/.env -f $week3 down

# Git Bash: provision the integrated platform
bash infra/week5/provision.sh --with-angular
```

## Import the repository collection

1. Open Postman Desktop and choose **Import** → **Files**.
2. Select `notification-api/Specification/Week4-Idempotency-Outbox.postman_collection.json`.
3. Open the imported collection and confirm `baseUrl` is `http://localhost:8080/api/v2`.

## Run the broker-outage sequence

**1** Pause only RabbitMQ. The API and SQL Server remain available.

```
docker compose -f infra/week5/compose.yml stop rabbitmq
```

**2** In Postman, use **Send** on requests 1, 2, and 3 in order. Do not edit the generated collection variables between requests.

| Request | Expected Postman evidence |
|---|---|
| 1 - Accept one durable bulk job | `202`; `jobId` is saved even though the broker is stopped. |
| 2 - Replay the same response | `202`; same job; `Idempotency-Replayed: true`. |
| 3 - Reject different payload | `409`; `application/problem+json`; no second job. |

> **Say this:** "The API guarantees durable SQL admission, not immediate broker delivery. Repeating the same operation is safe; reusing its key for a different operation is rejected."

# 6. Week 4: complete the Postman proof

1. Restart RabbitMQ and wait for it to become healthy in Docker Desktop.

```
docker compose -f infra/week5/compose.yml start rabbitmq
```

2. In Postman, open **Collection Runner**, select only **4 - Read the original job status**, choose one iteration, and run it. The collection script repeats that request while the relay backlog drains.

**Expected: zero failed tests and final status `Completed`, with total / processed / succeeded / failed = `1 / 1 / 1 / 0`.**

> `pm.execution.setNextRequest` works in Collection Runner, not the ordinary Send view. When presenting request 4 outside Runner, click **Send** repeatedly until it completes.

## Optional healthy-path rerun

With RabbitMQ healthy, run all four collection requests in order from Collection Runner. Request 1 generates a fresh idempotency key, so the rerun does not collide with the outage proof.

## Show the Next.js rendering modes in the browser

| Mode | Address | Explanation |
|---|---|---|
| SSG | `http://127.0.0.1:3000/rendering/ssg` | HTML produced at build time. |
| SSR | `http://127.0.0.1:3000/rendering/ssr` | HTML produced for each request. |
| ISR | `http://127.0.0.1:3000/rendering/isr` | Static output refreshed after revalidation. |
| CSR | `http://127.0.0.1:3000/rendering/csr` | The browser fetches after hydration. |

# 7. Week 5: platform, Vault, and UI proof

Run provisioning a second time to prove convergence, then use browser surfaces for the presentation.

```
# Git Bash - the second run is intentional
bash infra/week5/provision.sh --with-angular
```

Open the newest `infra/week5/.runtime/evidence/provision-*.txt` file in an editor. Point only to these sanitized lines:

```
vault_approle_paths=api:secret/nsa/api,worker:secret/nsa/worker,migrator:secret/nsa/migrator
vault_cross_role_reads=denied
secret_set_fingerprint_status=stable
```

| Surface | Address | Point to show |
|---|---|---|
| Swagger | `http://127.0.0.1:8080/swagger` | Versioned contract and documented response models. |
| Next.js | `http://127.0.0.1:3000` | Storefront, hooks, and rendering pages. |
| Angular | `http://127.0.0.1:4200` | `OnPush` demonstration. |
| RabbitMQ | `http://127.0.0.1:15672` | Topology and counts only; hide credentials. |
| Vault | `http://127.0.0.1:8200/v1/sys/health` | Health response only; never reveal bootstrap data. |

- API, Worker, and Migrator AppRoles can read only their own Vault paths.
- Applications receive rendered Docker secret files, never Vault tokens.
- .NET reads `/run/secrets` with KeyPerFile; only the Migrator owns schema changes.
- The second provision converges without discarding persisted volumes.

> Do not open anything below `infra/week5/.runtime/secrets`, `approle`, or `vault-init.txt`. Do not use `provision.sh --reset` or `docker compose down --volumes`.

# 8. Week 6: deploy two immutable pairs

Only the API/Next pair alternates behind Nginx. SQL, Vault, RabbitMQ, relay, and workers are shared. A meaningful rollback needs two distinct successful manifests.

> `--allow-dirty` is for a clearly labeled local rehearsal. A protected release uses a committed, attested hosted manifest.

## Create one reusable Postman health request

In Postman, create **GET - Active release** with URL `http://127.0.0.1:8088/health/version`. Save it and click **Send**. Keep the response open so the revision and color can be compared after each cutover.

1. Prepare and deploy release A in Git Bash.

```
bash infra/week6/prepare-local-release.sh --allow-dirty
# Use the exact manifest path printed by the preparation command:
bash infra/week6/deploy.sh <release-A-manifest-path>
```

Click **Send** in Postman. Record the returned `sha`, `color`, `X-Release-Sha`, and `X-Release-Color`.

2. Prepare and deploy a distinct release B.

```
bash infra/week6/prepare-local-release.sh --allow-dirty
bash infra/week6/deploy.sh <release-B-manifest-path>
```

Click **Send** again. The revision and active color must now match release B.

| Gate | Pass condition |
| --- | --- |
| Manifest | Exact content-addressed API, Next, and Migrator images. |
| Idle pair | Both healthy and reporting the expected revision before cutover. |
| Nginx candidate | Configuration validates before the atomic include switch. |
| Observation | Target-only traffic after cutover; former pair remains warm. |

# 9. Week 6: one-command rollback

1. Click **Send** on the saved Postman health request and confirm release B is active.
2. Run the rollback in Git Bash.

```
bash infra/week6/rollback.sh
```

3. Click **Send** again in Postman.

**Expected: the exact release A revision and color become active again. The response body and release headers must agree.**

## Rollback deliberately does not

- build an image;
- run or reverse a database migration;
- delete SQL, RabbitMQ, Vault, or release state; or
- retire the newly former color.

> **Layman's explanation:** two complete storefronts share one stockroom. Nginx changes which storefront sign customers follow. Rollback points the sign back to the checked previous storefront; it does not rewind the stockroom.

## If deployment or rollback fails

```
docker compose --env-file infra/week6/.runtime/release.env `
  -f infra/week6/compose.yml ps
```

Use Docker Desktop for container logs and open the newest file below `infra/week6/.runtime/evidence` in an editor. The scripts capture failed-candidate evidence and restore the prior Nginx include when a post-switch gate fails before state commit.

> Do not run `retire-previous.sh --confirm` during the demonstration. Keeping the former pair warm is what makes the rollback proof meaningful.

# 10. Presenter narrative

| Week | Visible evidence | Claim |
|---|---|---|
| 3 | Swagger shows `DeadLettered`, then `Completed`; RabbitMQ shows a retained DLQ command. | Transport is durable, retries are bounded, and recovery is controlled. |
| 4 | Postman shows 202 admission, exact replay, 409 conflict, and eventual completion across a broker outage. | SQL records intent atomically; Outbox and Inbox make interruption and redelivery safe. |
| 5 | Provisioning reruns; scoped AppRole evidence remains stable; each UI is healthy. | Infrastructure converges and secrets remain least-privileged. |
| 6 | Postman reports two distinct releases, then the exact former pair after rollback. | Traffic changes are health-gated and application rollback is reversible. |

## Architecture story

```
HTTP admission → SQL transaction → Outbox row → confirmed broker publish
        → manual-ACK consumer → Inbox receipt + business effect
```

- **Week 3:** separate process hosts isolate web traffic from background work.
- **Week 4:** durable admission closes the SQL-to-broker loss window and suppresses repeated effects.
- **Week 5:** Vault and Compose define a repeatable runtime and a narrow secret boundary.
- **Week 6:** paired releases change application traffic without pretending to reverse shared data.

> **Important boundary:** delivery is at least once. The system does not claim globally atomic SQL/RabbitMQ publication or exactly-once transport. Stable messages, an Outbox, and Inbox receipts make duplicate delivery harmless at the committed-effect boundary.

**Swagger** **Postman** **At-least-once** **Durable admission** **Bounded recovery** **Least privilege** **Reversible release**

# 11. Checklist and troubleshooting

| Proof | Pass condition |
|---|---|
| Week 3 DLQ | Swagger status is `DeadLettered`; RabbitMQ DLQ has one ready command while recovery is paused. |
| Week 3 recovery | Poison worker removed first; Swagger shows `Completed` and `2 / 2 / 2 / 0`; queues drain. |
| Week 4 idempotency | Postman shows same job plus replay header; changed payload gives RFC 7807 `409`. |
| Week 4 outage | Admission succeeds with RabbitMQ stopped; request 4 completes after restart with zero failed tests. |
| Week 5 rerun | Second provision succeeds; cross-role reads denied; secret fingerprint stable. |
| Week 6 rollback | Postman shows the exact prior revision and color with no shared-state reversal. |

## Fast diagnosis

| Symptom | Check |
|---|---|
| No Week 3 failure | Exact subject `[week3-poison]`; isolated worker healthy; canonical worker stopped. |
| DLQ entry disappears | Recovery worker was not paused, or a RabbitMQ fetch action consumed the evidence. |
| Replay is poisoned again | Remove the one-off poison worker before starting canonical recovery. |
| Postman request 4 does not loop | Use Collection Runner; ordinary Send does not honor `setNextRequest`. |
| Week 4 job stays Queued | RabbitMQ, Outbox Relay, and primary Worker health in Docker Desktop. |
| Rollback has no prior | Deploy two distinct successful manifests first. |

> **Close with:** "SQL records intent, the Outbox carries it, the Inbox suppresses repeated effects, unsafe messages are quarantined rather than lost, secrets remain scoped, and traffic can return to an exact prior application pair."

Reference: `docs/ADR-Week3.md`, `notification-api/README.md`, `infra/week5/README.md`, `infra/week6/README.md`, and `ExplanationWeek4To6.pdf`.